

C2SafeRust-LLM

Module 0: Data Stores — Line-by-Line Explanation

For Beginners | Mini Project S3

C2SafeRust-LLM — Module 0: Data Stores

Line-by-Line Code Explanation for Beginners

What is a Data Store? In this project, a "Data Store" is like a shared notebook that all the pipeline stages write to and read from. Instead of passing huge amounts of data between functions directly, each module fills in sections of these shared data structures. There are **two Data Stores**: - **D1** ([semantic_models.rs](#)) — Per-function analysis results (local to one function) - **D2** ([interprocedural_summaries.rs](#)) — Whole-program cross-function summaries

File 1: src/data_stores/mod.rs

(The Module Declaration File)

```
pub mod semantic_models;
pub mod interprocedural_summaries;
```

Line	Code	Explanation
1	<code>`pub mod semantic_models;`</code>	**Declares** the <code>`semantic_models`</code> sub-module and makes it **publicly accessible** to other parts of the project. The <code>`pub`</code> means "public — anyone can use this". The <code>`mod`</code> keyword introduces a module. The name <code>`semantic_models`</code> tells Rust to look for a file called <code>`semantic_models.rs`</code> in the same folder.
2	<code>`pub mod interprocedural_summaries;`</code>	Same idea — declares and exposes the <code>`interprocedural_summaries`</code> sub-module. Any module in the project can now import types from here using <code>`use`</code> .

Beginner Tip: In Rust, a `mod.rs` file in a folder acts like a table of contents. It lists all the files inside that folder that are part of this module. Without declaring a module here, Rust would simply not know that file exists.

File 2: src/data_stores/semantic_models.rs

(D1 — Per-Function Semantic Data Store)

Purpose: When Module 3 analyzes one C function, it records everything it learns (variables, how they're used, how memory is handled) into a `SemanticModel` struct. Think of it as a **"character sheet"** for a single function.

Line 1

```
use std::collections::HashMap;
```

- `use` — Brings something into scope so you can use its short name.
- `std` — Rust's standard library (built-in).
- `collections::HashMap` — A dictionary/map data structure that stores **key-value pairs** (e.g., `"variable_name" → "int"`).
- **Why we need it:** We'll use `HashMap` to store symbol tables and memory patterns, since we need to look up variables by their name quickly.

Lines 3–9: The `MemoryPattern Enum`

```
#[derive(Debug, Clone)]
pub enum MemoryPattern {
    Read,
    Write,
    Allocate,
    Free,
}
```

Line	Code	Explanation
<code>#[derive(Debug, Clone)]`</code>	Attribute macro	<code>Debug</code> — automatically generates code so you can print this enum with <code>{:?}</code> in debug output. <code>Clone</code> — lets you make a copy of this value when you need it. These are "auto-implemented traits" — Rust generates the code for you.
<code>pub enum MemoryPattern`</code>	Enum definition	<code>enum</code> stands for "enumeration" — a type that can be one of several named variants . This is like a traffic light that can be Red, Yellow, or Green. Here it can be <code>Read</code> , <code>Write</code> , <code>Allocate</code> , or <code>Free</code> .

Line	Code	Explanation
<code>`Read,`</code>	Variant	The variable is only **read** (e.g., <code>`printf("%d", x)`</code> reads <code>`x`</code>).
<code>`Write,`</code>	Variant	The variable is **written to** through a pointer (e.g., <code>`*ptr = 5`</code>).
<code>`Allocate,`</code>	Variant	Memory was allocated for this variable (e.g., <code>`ptr = malloc(100)`</code>).
<code>`Free,`</code>	Variant	Memory was freed (e.g., <code>`free(ptr)`</code>).

Why This Matters: Knowing whether a C variable is read-only, written-to, heap-allocated, or heap-freed is the key information needed to decide the correct Rust ownership type (`&T`, `&mut T`, or `Box<T>`).

Lines 11–15: The `PointerDepthClassification Enum`

```
#[derive(Debug, Clone)]
pub enum PointerDepthClassification {
    SingleLevel,
    MultiLevel { depth: usize },
}
```

Line	Code	Explanation
<code>`SingleLevel,`</code>	Variant	The variable is a simple pointer <code>`int*`</code> (one star). Rust handles this easily as a plain reference <code>`&T`</code> ; or <code>`&mut; T`</code> .
<code>`MultiLevel { depth: usize }`</code>	Struct-like Variant	The variable is a **double or deeper pointer** like <code>`int**`</code> or <code>`int***`</code> . The <code>`depth`</code> field stores the number of pointer levels (e.g., <code>`int**`</code> = <code>`depth: 2`</code>).

Why Multi-Level Pointers Matter in C-to-Rust Conversion: `int**` (a pointer-to-pointer) is extremely common in C for: - Output parameters: `void alloc_buf(int **out) { *out = malloc(100); }` — the caller gets a new heap allocation - Swap idioms: `void swap(int **a, int **b) { ... }` — modifies the caller's pointers Each case maps to a different Rust ownership type, so we must track the depth precisely.

Lines 17–21: The `UsePoint Struct`

```
#[derive(Debug, Clone)]
pub struct UsePoint {
    pub line: usize,
    pub column: usize,
}
```

Line	Code	Explanation
<code>`pub struct UsePoint`</code>	Struct definition	A <code>`struct`</code> groups related fields together (like a record or a row in a table). <code>`UsePoint`</code> stores the location where a variable is used.
<code>`pub line: usize,`</code>	Field	<code>`usize`</code> is an unsigned integer (can't be negative). This is the line number in the C source code where the variable appears.
<code>`pub column: usize,`</code>	Field	The column position on that line where the variable name starts.

Use in Def-Use Chains: A "def-use chain" tracks every place a variable is defined (assigned) or used (read). `UsePoint` is one entry in that chain, representing one specific occurrence. Example: If `ptr` appears on line 5, column 10, we store `UsePoint { line: 5, column: 10 }`.

Lines 23–28: The `UnionTransform Struct`

```
#[derive(Debug, Clone)]
pub struct UnionTransform {
    pub union_name: String,
    pub enum_name: String,
    pub local_variant_resolved: Option<String>,
}
```

Background: C **unions** allow multiple fields to share the same memory location. This is **inherently unsafe** in Rust. We must convert them to Rust **enums** which store only **ONE** variant at a time safely.

Line	Code	Explanation
<code>`pub union_name: String`</code>	Field	The name of the original C union (e.g., <code>"Data"</code>). <code>`String`</code> is Rust's heap-allocated text type.
<code>`pub enum_name: String`</code>	Field	The name of the generated Rust enum (e.g., <code>"DataEnum"</code>). We just capitalize and append "Enum".
<code>`pub local_variant_resolved: Option`</code>	Field	<code>`Option`</code> means the value is either <code>`Some("variant_name")`</code> or <code>`None`</code> . If we can tell locally which union field is active (e.g., <code>`u.i = 42`</code> means the <code>`i`</code> field is active), we store <code>`Some("i")`</code> . If we can't determine it locally, we store <code>`None`</code> .

Lines 30–34: The `UnresolvedUnionCase Struct`

```
#[derive(Debug, Clone)]
pub struct UnresolvedUnionCase {
    pub union_name: String,
    pub function_name: String,
}
```

Line	Code	Explanation
<code>`pub union_name: String`</code>	Field	Which union couldn't be resolved (e.g., <code>`"Data"`</code>).
<code>`pub function_name: String`</code>	Field	Which function has the unresolved case (e.g., <code>`"read_val"`</code>).

*This is a **flag to Module 4**: "Hey, function `read_val` uses union `Data` but we don't know which variant is active. Please do cross-function analysis to figure it out."*

Lines 36–46: The Main `SemanticModel Struct` (The D1 Store)

```
/// D1 Data Store: Per-Function Semantic Model
#[derive(Debug, Clone)]
pub struct SemanticModel {
    pub function_name: String,
    pub symbol_table: HashMap<String, String>,
    pub def_use_chains: HashMap<String, Vec<UsePoint>>,
    pub memory_patterns: HashMap<String, MemoryPattern>,
    pub pointer_depth_classifications: HashMap<String, PointerDepthClassification>,
    pub local_union_transforms: Vec<UnionTransform>,
    pub unresolved_union_cases: Vec<UnresolvedUnionCase>,
}
```

Field	Type	What it Stores
<code>`function_name`</code>	<code>`String`</code>	The name of the C function this model belongs to, e.g., <code>`"swap"`</code> .
<code>`symbol_table`</code>	<code>`HashMap`</code>	Maps variable names to their C types. E.g., <code>`"ptr" → "int*"`</code> , <code>`"x" → "int"`</code> .
<code>`def_use_chains`</code>	<code>`HashMap`</code>	For each variable, stores a list of every location (line+column) where it appears in the code.
<code>`memory_patterns`</code>	<code>`HashMap`</code>	For each variable, records the most significant memory operation (<code>`Read`</code> , <code>`Write`</code> , <code>`Allocate`</code> , or <code>`Free`</code>).
<code>`pointer_depth_classifications`</code>	<code>`HashMap`</code>	For each parameter, stores whether it's a single-level or multi-level pointer.
<code>`local_union_transforms`</code>	<code>`Vec`</code>	C unions successfully converted to Rust enum variants within this function.

Field	Type	What it Stores
<code>`unresolved_union_cases`</code>	<code>`Vec`</code>	C unions that couldn't be resolved locally — forwarded to Module 4.

Lines 48–60: The `SemanticModel::new()` Constructor

```
impl SemanticModel {
    pub fn new(function_name: String) -> Self {
        Self {
            function_name,
            symbol_table: HashMap::new(),
            def_use_chains: HashMap::new(),
            memory_patterns: HashMap::new(),
            pointer_depth_classifications: HashMap::new(),
            local_union_transforms: Vec::new(),
            unresolved_union_cases: Vec::new(),
        }
    }
}
```

Line	Code	Explanation
<code>`impl SemanticModel`</code>	Implementation block	Adds methods (functions) to the struct. Like adding behavior to a class.
<code>`pub fn new(function_name: String) -> Self`</code>	Constructor	<code>`new()`</code> is the Rust convention for constructors. It takes the function name and returns a fresh <code>`SemanticModel`</code> .
<code>`symbol_table: HashMap::new()`</code>	Initialize empty map	Creates an empty hash map. We'll fill it in during semantic analysis.
<code>`local_union_transforms: Vec::new()`</code>	Initialize empty vector	Creates an empty dynamic array. Entries will be pushed in as unions are encountered.

File 3: `src/data_stores/interprocedural_summaries.rs`

(D2 — Whole-Program Interprocedural Data Store)

Purpose: After Module 3 fills in per-function models, Module 4 looks at how functions interact with each other across the entire program. Everything it learns goes into the `InterproceduralSummaries` struct — the D2 Data Store.

Lines 1–2: Imports

```
use std::collections::HashMap;
use petgraph::graph::Graph;
```

Line	Code	Explanation
<code>`use std::collections::HashMap;`</code>	Import	Standard dictionary map — same as before.
<code>`use petgraph::graph::Graph;`</code>	Import	<code>`petgraph`</code> is an external Rust crate (library) for graph data structures. A <code>`Graph`</code> stores nodes and edges — here, nodes are function names and edges are function calls. It's listed in <code>`Cargo.toml`</code> as a dependency.

Lines 4–9: The `CallSite` Struct

```
#[derive(Debug, Clone)]
pub struct CallSite {
    pub caller: String,
    pub callee: String,
    pub line: usize,
}
```

*This struct records one function calling another. It's used as **edge data** in the call graph.*

Field	Explanation
<code>`caller`</code>	The function that makes the call (e.g., <code>`"main"`</code>).
<code>`callee`</code>	The function being called (e.g., <code>`"swap"`</code>).
<code>`line`</code>	The source code line number where the call happens.

Example: When `main()` calls `swap(&px, &py)` on line 15, we create: `CallSite { caller: "main", callee: "swap", line: 15 }`

Lines 11–16: The `AliasingVerdict` Struct

```
#[derive(Debug, Clone)]
pub struct AliasingVerdict {
    pub arg1: String,
    pub arg2: String,
    pub aliases: bool,
}
```

What is aliasing? When two different variable names refer to the **same memory location**, they are "aliases". This is a fundamental safety problem in Rust — you cannot have two mutable references to the same memory at the same time.

Field	Explanation
<code>`arg1`</code>	The first argument at a call site, as a string (e.g., <code>`"&x;"`</code>).
<code>`arg2`</code>	The second argument at the same call site (e.g., <code>`"&x;"`</code>).
<code>`aliases`</code>	<code>`true`</code> if both arguments point to the same memory; <code>`false`</code> otherwise.

C Idiom 1 Example: `foo(&x, &x)` — both args are `&x`, so they alias. This struct captures that verdict.

Lines 18–23: The `OwnershipTransferType` Enum

```
#[derive(Debug, Clone)]
pub enum OwnershipTransferType {
    ConsumesOwnership, // Idiom 2 (callee frees caller's T**)
    CreatesOwnership,  // Idiom 3 (callee allocates T** out-param for caller)
    None,
}
```

This enum represents **how ownership of heap memory moves** when functions are called.

Variant	C Pattern	Rust Equivalent
<code>`ConsumesOwnership`</code>	<code>`free(*buf)`</code> — callee frees memory passed in as <code>`T**`</code>	The parameter should be <code>`Box`</code> (callee takes ownership and drops it).
<code>`CreatesOwnership`</code>	<code>`*out = malloc(n)`</code> — callee allocates memory into <code>`T**`</code> out-param	The out-parameter returns <code>`Box`</code> to caller.
<code>`None`</code>	No special transfer	Regular borrow.

Lines 25–29: The `OwnershipTransferSummary` Struct

```
#[derive(Debug, Clone)]
pub struct OwnershipTransferSummary {
    pub param_name: String,
    pub transfer_type: OwnershipTransferType,
}
```

Field	Explanation
<code>`param_name`</code>	Which parameter has a special ownership transfer (e.g., <code>`"buf"`</code> or <code>`"out"`</code>).
<code>`transfer_type`</code>	Which kind of transfer happens (<code>`ConsumesOwnership`</code> or <code>`CreatesOwnership`</code>).

Lines 31–36: The DataDepFlag Struct

```
#[derive(Debug, Clone)]
pub struct DataDepFlag {
    pub function_name: String,
    pub variable_name: String,
    pub reason: String,
}
```

Purpose: Flags variables whose memory operations depend on **runtime values** that cannot be determined at compile time. **Example:** `if (condition) { free(ptr); }` — we don't know at compile-time whether `ptr` will be freed. This is **Idiom 6** — the most complex case.

Field	Explanation
<code>`function_name`</code>	Which function contains the problematic variable.
<code>`variable_name`</code>	Which variable has data-dependent memory operations.
<code>`reason`</code>	A human-readable description of why it was flagged (used in reports).

Lines 38–45: The OwnershipKind Enum

```
#[derive(Debug, Clone)]
pub enum OwnershipKind {
    UniqueHeapOwner,      // Box<T>
    MutableBorrow,        // &mut T
    SharedBorrow,         // &T
    HeapArray,            // Vec<T>
    ConservativeFallback, // Option<Box<T>>
}
```

This is the **final output classification** for each variable after all analysis is done. It directly maps to a Rust type.

Variant	Rust Type	When Used
<code>`UniqueHeapOwner`</code>	<code>`Box`</code>	Variable owns heap memory exclusively (from <code>`malloc`</code> or Idiom 2/3).
<code>`MutableBorrow`</code>	<code>`&mut; T`</code>	Variable modifies data through a pointer without owning it.
<code>`SharedBorrow`</code>	<code>`&T;`</code>	Variable only reads data through a pointer.
<code>`HeapArray`</code>	<code>`Vec`</code>	Variable points to a dynamically-sized array of elements.
<code>`ConservativeFallback`</code>	<code>`Option>`</code>	Idiom 6: runtime-conditional ownership — wrapped in <code>`Option`</code> for safety.

Lines 47–55: The Main InterproceduralSummaries Struct (The D2 Store)

```
/// D2 Data Store: Interprocedural Summaries
#[derive(Debug, Clone)]
pub struct InterproceduralSummaries {
    pub call_graph: Graph<String, CallSite>,
    pub alias_verdicts: HashMap<String, Vec<AliasingVerdict>>,
    pub ownership_transfer_summaries: HashMap<String, OwnershipTransferSummary>,
    pub cross_function_union_resolutions: HashMap<String, String>,
    pub data_dependent_ownership_flags: Vec<DataDepFlag>,
}
```

Field	Type	What it Stores
<code>`call_graph`</code>	<code>`Graph`</code>	The whole-program call graph. Nodes contain function names, edges contain <code>`CallSite`</code> data. Built by Module 4.1.
<code>`alias_verdicts`</code>	<code>`HashMap`</code>	Keyed by <code>`"caller:callee:line"`</code> — for each call site, stores alias analysis results. Populated by Module 4.2.
<code>`ownership_transfer_summaries`</code>	<code>`HashMap`</code>	Keyed by <code>`"function:param"`</code> — records whether each <code>`T**`</code> parameter creates or consumes heap ownership. Populated by Module 4.3.
<code>`cross_function_union_resolutions`</code>	<code>`HashMap`</code>	Keyed by <code>`"function:union_name"`</code> — stores resolved active variant for cross-function unions. Populated by Module 4.4.
<code>`data_dependent_ownership_flags`</code>	<code>`Vec`</code>	All variables flagged as having runtime-conditional memory operations. Populated by Module 4.5.

Lines 57–67: The InterproceduralSummaries::new() Constructor

```
impl InterproceduralSummaries {
    pub fn new() -> Self {
        Self {
            call_graph: Graph::new(),
            alias_verdicts: HashMap::new(),
            ownership_transfer_summaries: HashMap::new(),
            cross_function_union_resolutions: HashMap::new(),
            data_dependent_ownership_flags: Vec::new(),
        }
    }
}
```

Line	Code	Explanation
<code>`impl InterproceduralSummaries`</code>	Implementation block	Adds methods to the struct.
<code>`call_graph: Graph::new()`</code>	Initialize empty graph	Creates an empty graph with no nodes or edges. Module 4 will add nodes (functions) and edges (call sites) during analysis.
<code>`alias_verdicts: HashMap::new()`</code>	Initialize empty map	All fields start empty and are filled by Module 4 sub-analyses.

How D1 and D2 Flow Together

Module 3 Semantic Analysis
v Fills in D1 for each function
D1: SemanticModel { symbol_table, def_use_chains, memory_patterns, ... }
Module 4 Interprocedural Analysis
Reads D1 from all functions
v Fills in D2
D2: InterproceduralSummaries { call_graph, alias_verdicts, ownership_transfer_summaries, ... }
Module 5 Code Generation
Reads both D1 and D2
v
Infers ownership types -> Generates candidate Rust code

Key Rust Concepts Used in This Module

Concept	Example	Meaning
<code>`pub`</code>	<code>`pub struct SemanticModel`</code>	Public visibility — accessible from other modules.
<code>`enum`</code>	<code>`MemoryPattern::Free`</code>	A type that is one of several named variants.
<code>`struct`</code>	<code>`UsePoint { line, column }`</code>	Groups named fields together.
<code>`Option`</code>	<code>`Option`</code>	A value that is either <code>`Some(value)`</code> or <code>`None`</code> (no null pointers!).
<code>`HashMap`</code>	<code>`HashMap`</code>	A dictionary — look up a <code>`V`</code> value using a <code>`K`</code> key.
<code>`Vec`</code>	<code>`Vec`</code>	A growable list/array.
<code>`#[derive(...)]`</code>	<code>`#[derive(Debug, Clone)]`</code>	Auto-generate trait implementations.
<code>`impl Type { fn new() }`</code>	<code>`SemanticModel::new(...)`</code>	Rust's constructor pattern.

Summary

The **Data Stores module** defines the two central data structures that act as the backbone of the entire C2SafeRust-LLM pipeline:

SemanticModel (D1) — A per-function notebook capturing symbol tables, memory access patterns, pointer depths, and union transforms. Created fresh for each function and filled in by Module 3.

InterproceduralSummaries (D2) — A whole-program summary capturing the call graph, aliasing verdicts, ownership transfer summaries, cross-function union resolutions, and data-dependent ownership flags. Filled in by Module 4 and consumed by Modules 5, 6, and 7.

No analysis logic lives here — *this file only defines the shapes of the data. All the logic that populates these structures lives in Modules 3 and 4.*

Document generated for C2SafeRust-LLM Mini Project S3

Module 0: Data Stores — Complete Line-by-Line Explanation